02 home

02 — Home con datos del JWT (post-login)

> Prerrequisito: `docs/01_login.md` funcionando (login 200 → `isSuccess`).
> Objetivo: al entrar, mostrar en Home lo que trae el `access_token` (JWT) sin pedir nada más al backend.
> Restricciones: `docs/caso.md` §5.1 (solo datos ficticios, MVP sin publicación).

0. Qué trae el JWT (ejemplo real de forma)

Payload decodificado (la parte del medio del token):

```
{
  "username": "claudiodcv",
  "roles": ["superadmin"],
  "iss": "https://172-233-15-71.ip.linodeusercontent.com",
  "sub": "1",
  "iat": 1791481004,
  "exp": 1791484604
}
```

Qué significa cada claim:

Claim	Qué es en simple
`username`	Usuario logueado (vale username o email según `api.md`). Lo mostramos como título.
`roles`	Lista de roles (`superadmin`, ...). Define qué puede ver/hacer. En Home la mostramos como texto/chips.
`iss`	Emisor que firmó el token (tu backend). Sirve para verificar que viene del servidor correcto.
`sub`	Sujeto = id del usuario (1). Es el identificador para pedir sus eventos más adelante.
`iat`	Issued-at: cuándo se emitió (epoch segundos). Lo formateamos a fecha legible.
`exp`	Expiración (epoch segundos). Aquí `exp - iat = 3600` = 1 hora, igual que `expires_in` de `api.md`. Si `now >= exp`, hay que volver a login.

> Decodificar ≠ verificar. En el cliente decodificamos el payload **solo para mostrar**. La verificación real de firma y expiración se hace en el backend en cada request. Nunca confíes en roles del cliente para seguridad.

Paso 1 — Modelo + decodificador (sin librería extra)

No agregamos dependencia JWT: el payload es `Base64URL(JSON)`, lo decodificamos con `android.util.Base64` + Gson (Gson ya lo trae el Paso 1 del 01).

Estructura nueva:

```
core/jwt/
  SessionUser.kt   // data class con los claims
  JwtDecoder.kt    // object que decodifica access_token -> SessionUser
```

Archivo: `core/jwt/SessionUser.kt`

```kotlin
package com.rojas.syncpass.core.jwt

// data class = solo datos comparables/copiables (ver 01 Paso 3). Calca los claims del
 JWT.
// roles con default vacío para no crashear si el backend no lo manda.
data class SessionUser(
    val username: String = "",
    val roles: List<String> = emptyList(),
    val iss: String = "",
    val sub: String = "",
    val iat: Long = 0,
    val exp: Long = 0
) {
    // Derivado: segundos restantes. Negativo = expirado.
    val secondsRemaining: Long get() = exp - (System.currentTimeMillis() / 1000)
    val isExpired: Boolean get() = secondsRemaining <= 0
}
```

Archivo: `core/jwt/JwtDecoder.kt`

```kotlin
package com.rojas.syncpass.core.jwt

import android.util.Base64
import com.google.gson.Gson

// object = singleton sin estado: una sola función pura decode(), sin dependencias.
// Es object (no class) porque no necesita instancia, configuración ni ciclo de vida.
object JwtDecoder {
    private val gson = Gson()

    // Un JWT = header.payload.signature separados por '.'. Solo nos interesa [1]
 (payload).
    // Base64.URL_SAFE = alfabeto URL (-_ en vez de +/) y sin padding extra.
    // runCatching = si el token viene roto, devuelve failure en vez de crashear (ver
 01 AuthRepository).
    fun decode(accessToken: String): Result<SessionUser> = runCatching {
        val parts = accessToken.split(".")
        require(parts.size == 3) { "Token sin formato JWT" }
        val json = String(Base64.decode(parts[1], Base64.URL_SAFE), Charsets.UTF_8)
        gson.fromJson(json, SessionUser::class.java)
    }
}
```

Por qué no `interface` ni `class`: no hay contrato que otro implemente ni dependencias que inyectar en tests para este MVP. Si mañana verificas firma con clave pública, ahí sí pasaría a `class JwtVerifier(key)` testeable.

Paso 2 — Pasar el token de Login a Home (cambio mínimo al 01)

Hoy `LoginViewModel` solo marca `isSuccess = true` y el token se pierde. Cambio:

Archivo: `ui/login/LoginUiState.kt` — agregar:

```kotlin
data class LoginUiState(
    // ... lo que ya tienes ...
    val authToken: String? = null, // <- agregado 02: guarda el access_token del 200
    val isSuccess: Boolean = false
)
```

Archivo: `ui/login/LoginViewModel.kt` — en `onSuccess`:

```
        .onSuccess { response ->
            // Antes: it.copy(isLoading = false, isSuccess = true)
            // Ahora: además guarda el token para llevarlo a Home (memoria; DataStore va en 03).
            _uiState.update {
                it.copy(isLoading = false, isSuccess = true, authToken = response.access_token)
            }
        }
```

Archivo: `ui/login/LoginScreen.kt` — el callback ahora lleva el token:

```
        // Antes: onLoginSuccess: () -> Unit
        // Ahora: onLoginSuccess: (token: String) -> Unit
        @Composable
        fun LoginScreen(
            viewModel: LoginViewModel = viewModel(),
            onLoginSuccess: (token: String) -> Unit = {}
        ) { /* ... */ }

        @Composable
        fun LoginContent(
            state: LoginUiState,
            // ... lo que ya tienes ...
            onLoginSuccess: (token: String) -> Unit = {}
        ) {
            LaunchedEffect(state.isSuccess, state.authToken) {
                val token = state.authToken
                if (state.isSuccess && !token.isNullOrBlank()) onLoginSuccess(token)
            }
            // ... resto igual ...
        }
```

Paso 3 — Home: estado + ViewModel

```
        ui/home/
          HomeUiState.kt       // data class inmutable
          HomeViewModel.kt     // class : ViewModel, decodifica el token
          HomeScreen.kt        // stateful + content stateless + preview sin ViewModel
```

Archivo: `ui/home/HomeUiState.kt`

```
        package com.rojas.syncpass.ui.home

        import com.rojas.syncpass.core.jwt.SessionUser

        // data class inmutable = una sola fuente de verdad para la Home (ver 01: inmutabilidad
         para UDF).
        data class HomeUiState(
            val user: SessionUser? = null,
            val decodeError: String? = null
        )
```

Archivo: `ui/home/HomeViewModel.kt` — aquí viven DOS clases juntas (mismo archivo, mismo package): 1) `HomeViewModel` (el dueño del estado) y 2) `HomeViewModelFactory` (su receta de construcción, al final del archivo). Van juntas porque la factory existe solo para este ViewModel; separarla en otro archivo sería ruido. Alternativa válida: `ui/home/HomeViewModelFactory.kt` aparte si el equipo prefiere un archivo por clase — el código es idéntico, solo cambia el archivo.

```
package com.rojas.syncpass.ui.home

import androidx.lifecycle.ViewModel
import androidx.lifecycle.ViewModelProvider // <- import agregado para la Factory
import com.rojas.syncpass.core.jwt.JwtDecoder
import kotlinx.coroutines.flow.MutableStateFlow
import kotlinx.coroutines.flow.StateFlow
import kotlinx.coroutines.flow.asStateFlow
import kotlinx.coroutines.flow.update

// class : ViewModel con parámetro token: una instancia por Home, sobrevive a rotación.
// Es class (no object) porque cada login tiene un token distinto; un object filtraría
 usuarios.
class HomeViewModel(accessToken: String) : ViewModel() {
    // Backing property igual que en login: privada mutable, pública solo lectura.
    private val _uiState = MutableStateFlow(HomeUiState())
    val uiState: StateFlow<HomeUiState> = _uiState.asStateFlow()

    init {
        // Se decodifica una vez al crear. Sin corrutina: es CPU local, no red.
        JwtDecoder.decode(accessToken)
            .onSuccess { user -> _uiState.update { it.copy(user = user) } }
            .onFailure { _uiState.update { it.copy(decodeError = "Sesión inválida.
 Vuelve a ingresar.") } }
    }
}
}

// ---- Va en el MISMO archivo HomeViewModel.kt, debajo del ViewModel (no en
 HomeScreen.kt,
// no en MainActivity): la factory se define donde vive lo que construye. Quien la USA
 es
// HomeScreen.kt con viewModel(factory = HomeViewModelFactory(accessToken)) (ver Paso
 4).
// class : ViewModelProvider.Factory porque DEBE guardar el token y crear ViewModels.
```

```
// No es object: cada Home tiene su token. No es data class: no se compara, solo
 construye.
class HomeViewModelFactory(private val accessToken: String) : ViewModelProvider.Factory
 {
    @Suppress("UNCHECKED_CAST") // seguro: esta factory solo se usa con HomeViewModel
    override fun <T : ViewModel> create(modelClass: Class<T>): T {
        return HomeViewModel(accessToken) as T // aquí SÍ usamos el constructor con
 parámetro
    }
}
```

Corrección aplicada (crash real en clase): `HomeScreen` usaba `viewModel()` sin factory para un `HomeViewModel(accessToken)` con parámetro. El factory por defecto solo crea ViewModels sin argumentos, así que al entrar a Home la app cerraba (`Cannot create an instance of class HomeViewModel`). Si ves ese error en Logcat, siempre es factory faltante ante ViewModel con parámetros.

Por qué hace falta un Factory (explicación para estudiantes)

El problema: tú no creas el ViewModel con `HomeViewModel(token)` a mano (eso rompería el scope que sobrevive a rotación). Lo crea **el sistema** cuando llamas `viewModel()`. Pero el sistema solo sabe hacer `HomeViewModel()` **sin argumentos** (como con `LoginViewModel`, que tiene default o ningún parámetro). Si tu ViewModel exige un parámetro obligatorio (`accessToken`), el sistema no sabe qué pasarle y crashea.

La solución — Factory = 'receta de construcción': le entregas al sistema un objeto que sí sabe construirlo. Es el patrón Factory clásico: en vez de decir *cómo* con `new`, entregas *quién sabe hacerlo*:

Y en la pantalla (`ui/home/HomeScreen.kt`, NO en el archivo del ViewModel) se lo pasas al sistema:

```
val viewModel: HomeViewModel = viewModel(factory = HomeViewModelFactory(accessToken))
```

Cómo funciona por dentro, paso a paso:

1 Primera vez que se dibuja `HomeScreen`, `viewModel(factory)` pregunta al `ViewModelStore`: '¿ya hay un HomeViewModel?' No hay → llama a `factory.create(HomeViewModel::class.java)` → nace con tu token → se guarda en el store.

1 Rotas el teléfono: la Activity se destruye, pero el store se conserva → la nueva pantalla pide `viewModel(factory)` → ya existe → **reutiliza el mismo**, la factory no se usa de nuevo.

1 Haces logout y el token cambia: como la factory es distinta, conviene usar `viewModel(factory, key = accessToken)` para que cada sesión tenga su instancia y no se reutilice el usuario anterior.

Regla para decidir siempre: ¿tu ViewModel tiene algún parámetro sin default (token, id, repository sin default)? → necesita Factory. ¿No tiene parámetros o todos tienen default (como `LoginViewModel`)? → `viewModel()` solo basta.

Paso 4 — Home: pantalla reutilizando `AuthCard`

Archivo: `ui/home/HomeScreen.kt`

```
package com.rojas.syncpass.ui.home

import androidx.compose.foundation.layout.Arrangement
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.foundation.layout.padding
import androidx.compose.material3.Button
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.runtime.collectAsState
import androidx.compose.runtime.getValue
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.tooling.preview.Preview
import androidx.compose.ui.unit.dp
import com.rojas.syncpass.core.jwt.SessionUser
import com.rojas.syncpass.ui.components.AuthCard
import com.rojas.syncpass.ui.theme.SyncPassTheme
import java.text.SimpleDateFormat
import java.util.Date
import java.util.Locale

// Stateful: pide el ViewModel CON su factory (lleva el token). Sin factory crashea al
 entrar.
// Ver HomeViewModelFactory en Paso 3 y nota de la corrección.
@Composable
fun HomeScreen(accessToken: String, onLogout: () -> Unit = {}) {
    val viewModel: HomeViewModel = androidx.lifecycle.viewmodel.compose.viewModel(
        factory = HomeViewModelFactory(accessToken)
    )
    val state by viewModel.uiState.collectAsState()
    HomeContent(state = state, onLogout = onLogout)
}
```

```
    // Stateless: dibuja según HomeUiState. Es lo que usa el preview con datos ficticios.
    @Composable
    fun HomeContent(state: HomeUiState, onLogout: () -> Unit = {}) {
        Column(
            modifier = Modifier.fillMaxSize().padding(16.dp),
            verticalArrangement = Arrangement.Center,
            horizontalAlignment = Alignment.CenterHorizontally
        ) {
            AuthCard(title = "Bienvenido") {
                // Todo esto es el children (ColumnScope) del Card, igual que en login.
                val user = state.user
                if (user != null) {
                    // Text con username + id: lo que el JWT dice que eres.
                    Text("Usuario: ${user.username}", style =
     MaterialTheme.typography.titleMedium)
                    Text("ID (sub): ${user.sub}", style =
     MaterialTheme.typography.bodyMedium)
                    // roles: lista del claim. En MVP como texto; luego como chips/filtros.
                    Text(
                        "Roles: ${user.roles.joinToString()}",
                        style = MaterialTheme.typography.bodyMedium
                    )
                    Text("Emisor: ${user.iss}", style = MaterialTheme.typography.bodySmall)
                    // iat/exp vienen en epoch-segundos: se formatean a fecha legible solo
     para mostrar.
                    Text("Emitido: ${formatEpoch(user.iat)}", style =
     MaterialTheme.typography.bodySmall)
                    Text("Expira: ${formatEpoch(user.exp)}", style =
     MaterialTheme.typography.bodySmall)
                    if (user.isExpired) {
                        Text("Sesión expirada, vuelve a ingresar.", color =
     MaterialTheme.colorScheme.error)
                    }
                } else {
```

```
                    Text(state.decodeError ?: "Cargando sesión...")
                }
                Button(onClick = onLogout, modifier = Modifier.fillMaxWidth()) {
                    Text("Cerrar sesión")
                }
            }
        }
    }

    // Helper solo de presentación: epoch-segundos -> fecha local. No es lógica de negocio.
    private fun formatEpoch(epochSeconds: Long): String {
        if (epochSeconds <= 0) return "-"
        val fmt = SimpleDateFormat("dd-MM-yyyy HH:mm", Locale.getDefault())
        return fmt.format(Date(epochSeconds * 1000))
    }

    // Preview SIN token real ni ViewModel: SessionUser ficticio directo (caso §5.1).
    @Preview(showBackground = true)
    @Composable
    private fun HomeContentPreview() {
        SyncPassTheme {
            HomeContent(
                state = HomeUiState(
                    user = SessionUser(
                        username = "claudiodcv",
                        roles = listOf("superadmin"),
                        iss = "https://172-233-15-71.ip.linodeusercontent.com",
                        sub = "1",
                        iat = 1791481004,
                        exp = 1791484604
                    )
                )
            )
        }
    }
```

Paso 5 — Navegación mínima en `MainActivity` (sin librería)

Para el MVP no agregamos Navigation-Compose: dos rutas en memoria.

```
// MainActivity.kt
import androidx.compose.runtime.getValue
import androidx.compose.runtime.mutableStateOf
import androidx.compose.runtime.saveable.rememberSaveable
import androidx.compose.runtime.setValue
import com.rojas.syncpass.ui.home.HomeScreen
import com.rojas.syncpass.ui.login.LoginScreen

class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        ...
        setContent {
            SyncPassTheme {
                // rememberSaveable = sobrevive a rotación (detalle abajo). Tipo
 explícito String?
                // para que el delegado infiera bien (sin esto: TypeVariable/Any).
                var token: String? by rememberSaveable { mutableStateOf<String?>(null) }
                // Copia a val local: el smart cast SÍ funciona en locales, NO en
 delegadas.
                // Por eso NO se usa 'token!!': se usa currentToken ya acotado a String.
                val currentToken: String? = token
                if (currentToken == null) {
                    LoginScreen(onLoginSuccess = { t -> token = t })
                } else {
                    HomeScreen(accessToken = currentToken, onLogout = { token = null })
                }
            }
        }
    }
}
```

Errores que evita esta forma (los que salieron en clase):
1. *Property delegate must have getValue/setValue*: faltaban `import getValue` y `import setValue`.
by los exige (`getValue` para leer, `setValue` extra para `var`). Las 4 líneas de imports de arriba son obligatorias.
2. *Smart cast impossible / type mismatch Any vs String*: `token` es propiedad **delegada**, el compilador no puede acotarla con `if (token == null)` ni inferir su tipo solo (`var token by ...` sin tipo da TypeVariable/Any). Solución doble: tipo explícito `String?` en la declaración + copia a `val currentToken` local, que sí admite smart cast y llega al `else` como `String` no-nulo sin `!!`.

>

Cómo recuerda el token de una vista a otra: `token` no vive en Login ni en Home, vive **arriba** de ambas, en el `setContent` de MainActivity. Flujo: login OK → `onLoginSuccess(t)` escribe `token = t` → el estado cambia → Compose **recompone** el `setContent` → ahora `currentToken != null` → se dibuja `HomeScreen(token)` en vez de `LoginScreen`. Logout hace `token = null` y se vuelve al login. Es navegación por estado: no hay librería, solo un `if` sobre un estado recordado. `rememberSaveable` además lo conserva ante **rotación** (lo guarda en el SavedState de la Activity); ante muerte de proceso el Bundle solo conserva tipos simples (String sí) y ante cierre total se pierde (la persistencia real con DataStore va en el 03).

>

Cuando haya 3+ destinos (eventos, QR, encuestas) migrar a `navigation-compose`. Para login→home el `if` es suficiente y evita una dependencia.

Paso 6 — Probar

1. Login con `juan_perez / Password123` → debe navegar a Home y mostrar `username`, `roles`, `sub`, `iss`, fechas de `iat/exp`.
2. Compara con el payload que pegaste: mismo `username`, `roles=[superadmin]`, `exp-iat=3600`.
3. Token roto (pega `abc.def` en preview/fake): debe mostrar `Sesión inválida`.
4. Expirado: si `isExpired` es true, muestra aviso y pide re-login (el control real lo hará el backend con 401).
5. Rotación en Home: los datos siguen (viven en `HomeViewModel`, no en `remember`).
6. Claro/oscuro: `AuthCard` ya lo hereda, verifica ambos.

Checklist 02

- [ ] `JwtDecoder` decodifica el `access_token` real del login sin crash con token roto.
- [ ] Home muestra `username`, `roles`, `sub`, `iss`, `iat/exp` formateados desde el JWT, sin request extra.
- [ ] `LoginContentPreview` y `HomeContentPreview` usan solo datos ficticios, sin `LoginViewModel()` construido en preview.
- [ ] Navegación login→home y logout funcionan; rotación no pierde sesión en memoria.
- [ ] Ningún secreto nuevo en duro; el token vive en memoria (persistencia segura va en 03).

Nota — Cerrar sesión: qué hace hoy y qué falta

Hoy el logout **sí funciona para el MVP**, no es un botón muerto. Cadena completa:

1. `HomeContent` tiene `Button(onClick = onLogout)` → llama el `onLogout` que recibió por parámetro.
2. `HomeScreen` lo reenvía tal cual (`HomeContent(state, onLogout)`), sin lógica propia.
3. `MainActivity` lo definió como `onLogout = { token = null }` → borra el token en memoria →

Compose recompone el `setContent` → `currentToken == null` → vuelve a `LoginScreen`.

Pruébalo: presiona "Cerrar sesión" y debes volver al login; rota antes y después, igual funciona. Lo que **falta** (y por eso parece "a medias") es el logout *real* con persistencia, que va en el `03_session.md`: borrar el `access_token` guardado en DataStore, cancelar sincronizaciones pendientes y, si el backend tiene endpoint de logout/invalidación, llamarlo. Con solo memoria, al matar la app la sesión ya se pierde sola; con DataStore habrá que borrarla explícito.

Pasos siguientes (hacia el offline-first del caso §3.4)

03 — Sesión persistente (DataStore)

Hoy el token vive en `rememberSaveable`: sobrevive a rotación pero muere con la app. El 03 guarda el `access_token` en **DataStore** (clave-valor asíncrono, reemplazo moderno de SharedPreferences): auto-login al abrir, logout que borra de verdad, y control de `exp` (si el token expiró o el backend responde 401, se vuelve al login solo).

04 — Room: la base local (qué es y qué hará)

Qué es Room: capa de persistencia local sobre SQLite. No escribes SQL a mano para lo típico: defines `@Entity` (tablas), `@Dao` (consultas) y `@Database` (la base). Es el "modo offline-first" que exige el caso §2.4/§3.4: la app debe funcionar al 100% sin señal en las unidades vecinales y sincronizar después.

Qué guardará (del caso §3.2):

Entity	Qué es en simple	Clave para sync
`Proyecto`	Ficha del proyecto comunitario (título, área, unidad vecinal, fechas, estado)	`id` servidor + `updatedAt`
`Evento`	Taller/actividad (proyecto asociado, fecha/hora, cupos, recinto, coordenadas)	`id` + descarga periódica del backend
`Asistencia`	Cada marcación QR (evento, asistente, timestamp, validación)	`is_synced: Boolean` + `timestamp`
`Encuesta`	Valoración 1-5 + utilidad + comentario + fecha de envío	`is_synced: Boolean`

Cómo funcionará: en terreno todo se escribe a Room (lectura QR → `Asistencia(is_synced=false)`). Al recuperar señal, un worker sube por lotes (`batch sync`) lo no sincronizado vía Retrofit, marca `is_synced=true` con sus `timestamps`, y descarga catálogo nuevo (sincronización bidireccional del caso §3.4). El login/Home de este 02 no cambia: solo gana que la Home mostrará pendientes de sincronización y la sesión vendrá de DataStore en vez de memoria.

Contrato: `docs/api.md`. *Contexto:* `docs/caso.md`. *Base UI:* `docs/01_login.md`.